Rapport de TP - LAB 1

Conception et test d'une API REST avec HTTP et Postman

Module : Développement Backend - Master IAIDDWM
Réalisé par : Khaoula BAHLOUL
Enseignant : Pr Mohamed LACHGAR
Novembre 2026

1. Introduction

Dans ce TP, j'ai utilisé Postman pour tester une API REST publique appelée dummyjson.com. Cette API simule un backend de gestion de produits. Le but était de comprendre le protocole HTTP et l'architecture REST, de bien utiliser Postman, et de manipuler les méthodes HTTP, les headers, les paramètres de requête (query params) et les bodies JSON.

2. Outils utilisés

- Postman (pour créer et envoyer les requêtes)
- curl dans PowerShell (pour un premier test)
- Un navigateur web
- L'API de démonstration https://dummyjson.com

3. Captures d'écran

Les captures d'écran de Postman (requêtes, réponses, tests et environnement) sont regroupées dans un **dossier à part, fourni avec ce rapport**. Dans ce document, je décris ce que j'ai fait et ce que j'ai observé, et les captures correspondantes se trouvent dans ce dossier.

4. Travail réalisé

4.1 Premier test avec curl

J'ai d'abord envoyé une requête GET avec curl pour lister 5 produits : `curl.exe -X GET "https://dummyjson.com/products?limit=5"`. La réponse est en JSON. Elle contient un tableau **products** avec les produits, et aussi **total**, **skip** et **limit**. Ces trois champs servent à la pagination : l'API envoie les produits par petits groupes et pas tous en même temps.

4.2 Organisation de la collection Postman

J'ai créé une collection nommée **LAB1 - Gestion Produits REST** avec 4 dossiers : Lecture (GET), Création (POST), Modification (PUT/PATCH) et Suppression (DELETE). Chaque requête a été rangée dans le dossier qui correspond à sa méthode.

4.3 Requêtes GET (lecture)

- **Produit par id** : GET sur `/products/1`, la réponse est 200 OK avec le produit n°1.
- **Recherche** : GET sur `/products/search` avec le paramètre **q = phone** dans l'onglet Params. Postman a construit tout seul l'URL `/products/search?q=phone`.

4.4 Requête POST (création)

J'ai envoyé un POST sur `/products/add` avec le header `Content-Type: application/json` et un body en JSON (onglet Body, raw, JSON) : title, price, category et stock. Le serveur a répondu avec le produit créé et un nouvel **id (195)**. C'est une API de simulation, donc le produit n'est pas vraiment enregistré.

4.5 Requêtes PUT et PATCH (modification)

J'ai testé les deux méthodes sur `/products/1` :

- **PUT** remplace toute la ressource : j'ai envoyé tous les champs (title, price, category, stock).
- **PATCH** modifie seulement une partie : j'ai envoyé uniquement le champ price.

Dans les deux cas, la réponse est 200 OK.

4.6 Requête DELETE (suppression)

J'ai envoyé un DELETE sur `/products/1`. La réponse est 200 OK et elle contient `"isDeleted": true`. La suppression est seulement simulée : si on refait un GET sur le même id, le produit est toujours là.

4.7 Variables d'environnement

Pour ne pas écrire l'URL complète dans chaque requête, j'ai créé un environnement **Demo API Env** avec deux variables : **baseUrl** (https://dummyjson.com) et **productId** (1). Ensuite, j'ai remplacé les URLs par `{{baseUrl}}/products/{{productId}}`. Il faut penser à sélectionner l'environnement en haut à droite, sinon les variables ne marchent pas.

4.8 Scripts de test

Dans l'onglet Scripts (Post-response), j'ai écrit des tests en JavaScript avec `pm.test`. Sur la requête POST, le test vérifie le code de statut. Le script récupère aussi l'id du produit créé avec `pm.response.json()` et le stocke dans la variable d'environnement **createdProductId** avec `pm.environment.set`. Les résultats s'affichent dans l'onglet Test Results (PASS ou FAIL).

4.9 Cas d'erreur

- **404 Not Found** : GET sur `/products/99999`, un produit qui n'existe pas. J'ai écrit un test qui vérifie le code 404 et que le message d'erreur contient "not found".
- **400 Bad Request** : POST avec un JSON volontairement cassé (il manque l'accolade fermante). J'ai écrit un test qui vérifie que le code est 400.

4.10 Exercices en autonomie

- GET des produits de la catégorie smartphones avec `/products/category/smartphones`.
- POST d'un nouvel utilisateur sur `/users/add` avec firstName, lastName et age. L'API a répondu **201 Created** avec un nouvel id.
- Test du temps de réponse : `pm.expect(pm.response.responseTime).to.be.below(500)`, pour vérifier que l'API répond en moins de 500 ms.

5. Codes HTTP rencontrés

Code	Nom	Signification	Où je l'ai vu
200	OK	La requête a réussi	GET, PUT, PATCH, DELETE

Code	Nom	Signification	Où je l'ai vu
201	Created	Une ressource a été créée	POST (ajout d'un utilisateur)
400	Bad Request	Requête invalide (erreur du client, ex : JSON mal écrit)	Test avec le JSON cassé
404	Not Found	La ressource demandée n'existe pas	GET sur /products/99999

Pour retenir : les codes 2xx veulent dire que tout s'est bien passé, les codes 4xx sont des erreurs du côté du client, et les codes 5xx sont des erreurs du côté du serveur.

6. Difficultés rencontrées

- Au début, je n'ai pas trouvé l'onglet Collections dans la nouvelle interface de Postman.
- Pour l'export, Postman demande d'écrire une raison avant de lancer l'export.
- Il faut bien choisir l'environnement en haut à droite pour que les variables {{baseUrl}} fonctionnent.

7. Conclusion

Ce TP m'a permis de comprendre comment fonctionne une API REST et comment la tester avec Postman. J'ai appris à utiliser les méthodes GET, POST, PUT, PATCH et DELETE, à envoyer un body JSON, à utiliser des variables d'environnement et à écrire des tests simples en JavaScript. Je sais aussi lire les principaux codes de statut HTTP.

Livrables fournis

- La collection Postman exportée (fichier JSON)
- L'environnement Postman exporté (fichier JSON)
- Ce rapport PDF
- Le dossier contenant les captures d'écran